

 Playwright + TypeScript

Master Playwright Fixtures Like a Pro

Your ultimate visual guide — from zero to interview-ready.
Clear, visual & motivating!

TypeScript

Playwright

Interview Prep

💪 You've got this! Every expert was once a beginner. Read this once, understand deeply — ace your interview!



What is a Fixture?

The big picture — start here



A **Fixture** is a reusable **Setup Helper** that automatically prepares everything your test needs **before it runs**, and cleans everything up **after it finishes** — so you never write the same setup code twice.

🍽️ Think of it like having dinner...

**BEFORE TEST****Setup**

Set the table, place fork & knife, fill the water glass

**YOUR TEST****You Eat!**

This is your actual test running — the real action

**AFTER TEST****Teardown**

Clear the table, wash dishes, clean up everything

🔧 Setup + ▶ Test + 🧹 Teardown = **FIXTURE**



Without vs With Fixture

See the exact problem Fixtures solve

Imagine 5 tests all need a logged-in page. Without fixtures = repeat setup 5 times. With fixtures = write it once.

login.spec.ts ✗ WITHOUT Fixture

```
// 🤖 Repeating the SAME setup in EVERY single test
test('test 1', async () => {
  const browser = await chromium.launch();
  const page    = await browser.newPage();
  await page.goto('https://myapp.com');
  await page.fill('#user', 'admin');
  await browser.close(); // manual cleanup!
});
test('test 2', async () => {
  const browser = await chromium.launch(); // 📄 COPY-PASTE again!
  const page    = await browser.newPage();
  // 🤖 same setup AGAIN and AGAIN...
});
```

login.spec.ts ✓ WITH Fixture

```
import { test } from '@playwright/test';

// ✨ 'page' is BUILT-IN – Playwright handles everything!
test('test 1', async ({ page }) => {
  await page.goto('https://myapp.com');
  // ✓ Browser auto-created AND destroyed for you!
});
test('test 2', async ({ page }) => {
  await page.goto('https://myapp.com');
  // ✓ Zero repeated setup needed!
});
```

Without Fixture ✗

✗ Repeat setup in every test

✗ Hard to maintain & update

✗ Tests are long and messy

With Fixture ✓

✓ Write setup only once

✓ Update in one place

✓ Tests are clean & focused

Without Fixture ❌

❌ Manual cleanup required

❌ Copy-paste across files

With Fixture ✅

✅ Auto cleanup handled

✅ Share fixtures everywhere

**Built-in Fixtures**

What Playwright gives you for free

Playwright ships with these powerful fixtures — just destructure them in your test function!

**{ page }**

Opens a fresh browser page for each test. Your most-used fixture for all UI tests.

**{ browser }**

Launches the browser instance. Use when you need fine-grained browser control.

**{ context }**

Isolated browser context — like a private window with its own cookies & storage.

**{ request }**

Makes API requests directly without a browser UI. Perfect for API testing.

example.spec.ts

✅ Just add them!

```
test('demo', async ({ page, context, request }) => {  
  await page.goto('https://myapp.com'); // UI  
  const p2 = await context.newPage(); // 2nd tab  
  const res = await request.get('/api/users'); // API call  
});
```



Custom Fixture

Build your own reusable setup



Real-World Use Case

Every test needs the user to be logged in. Instead of writing login steps every time, create a `loggedInPage` fixture — all tests instantly get a pre-authenticated page!

1

SETUP — Before your test runs

Navigate to login page, fill credentials, click login. Runs automatically before every test using this fixture.

2

await use(page) — Your test runs HERE

The magic line. Everything before = setup. Everything after = teardown. Test gets the page in a ready, logged-in state.

3

TEARDOWN — After your test finishes

Logout, close connections, reset state. Playwright calls this automatically — even if the test fails!



fixtures.ts

Define your fixture

```
import { test as base, Page } from '@playwright/test';

export const test = base.extend<{ loggedInPage: Page }>({

  loggedInPage: async ({ page }, use) => {

    // —  SETUP —————
    await page.goto('https://myapp.com/login');
    await page.fill('#username', 'admin');
    await page.fill('#password', 'pass123');
    await page.click('#loginBtn');
    await page.waitForURL('**/dashboard');

    // —  TEST RUNS HERE —————
    await use(page);

    // —  TEARDOWN —————
    await page.click('#logout');
  },
});
```

dashboard.spec.ts

☒ Use the fixture

```
import { test } from './fixtures'; // ← import YOUR test!

// All 3 tests are already logged in ☒
test('view dashboard', async ({ loggedInPage }) => {
  await loggedInPage.click('#dashboard');
});
test('view profile', async ({ loggedInPage }) => {
  await loggedInPage.click('#profile');
});
test('view settings', async ({ loggedInPage }) => {
  await loggedInPage.click('#settings');
});
```

**SETUP**

Code before
await use()

**TEST RUNS**

await use(page)
← magic line

**TEARDOWN**

Code after
await use()



Interview Q&A

Top questions you'll likely be asked

Q What is a fixture in Playwright?

A A fixture is a reusable setup and teardown mechanism. It runs automatically before each test to prepare the environment and after each test to clean up — keeping tests isolated, independent, and DRY (Don't Repeat Yourself).

Q What does `await use()` do inside a fixture?

A `await use()` is the pivot point — code *before* it is setup (runs before the test), code *after* it is teardown (runs after the test, even if it fails).

Q Name the built-in fixtures Playwright provides.

A Four built-ins: `page` (fresh tab), `browser` (browser instance), `context` (isolated context), and `request` (API testing without browser UI).

Q Fixture vs `beforeEach` — what's the difference?

A `beforeEach` is local to one spec file. Fixtures are composable, shareable across files, support dependency injection, and can be scoped to test/worker/project level — the Playwright-recommended approach.

Q How do you create a custom fixture in Playwright TypeScript?

A Use `base.extend()` to extend the default test. Define your fixture as `async ({ page }, use) => {}`. Write setup, call `await use(value)`, then teardown. Export the extended `test` and import it in spec files.

Q What is fixture scope and why does it matter?

A `test` scope (default) creates a new instance per test. `worker` scope shares one instance across all tests in a worker — great for expensive setups like database connections.



Quick Cheat Sheet

Stick this on your wall!



SETUP

Code before
`await use()`



TEST RUNS

At the
`await use(x)` line



TEARDOWN

Code after
`await use()`



BUILT-IN

`page`, `browser`,
`context`, `request`



CUSTOM

`base.extend({})`
`async ({p}, use)`



SCOPE

`test` (default)
`worker` (shared)

★ KEY RULE TO REMEMBER

Always import `test` from your **fixtures file**, not from `@playwright/test` directly — otherwise your custom fixtures won't be available in your tests!



You're Ready to Ace That Interview!

Playwright + TypeScript | Interview Preparation Guide

Fixtures = Setup → Test → Teardown | `await use()` is the magic line

Study hard, stay confident, and go get that job! 💪